

Python

- ▶ **Python** is a general purpose, dynamic, high-level, and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.
- ▶ Python is *easy to learn* yet powerful and versatile scripting language, which makes it attractive for Application Development.
- ▶ Python's syntax and *dynamic typing* with its interpreted nature make it an ideal language for scripting and rapid application development.

- ▶ Python supports *multiple programming pattern*, including object-oriented, imperative, and functional or procedural programming styles.
- ▶ Python is not intended to work in a particular area, such as web programming. That is why it is known as *multipurpose* programming language because it can be used with web, enterprise, 3D CAD, etc.
- ▶ We don't need to use data types to declare variable because it is *dynamically typed* so we can write `a=10` to assign an integer value in an integer variable.

- ▶ Python makes the development and debugging *fast* because there is no compilation step included in Python development, and edit-test-debug cycle is very fast.

Python 2 vs. Python 3

- ▶ Python 2 and Python 3 are very much different from each other.
- 1. Python 2 uses **print** as a statement and used as `print "something"` to print some string on the console. On the other hand, Python 3 uses **print** as a function and used as `print("something")` to print something on the console.
- 2. Python 2 uses the function `raw_input()` to accept the user's input. It returns the string representing the value, which is typed by the user. To convert it into the integer, we need to use the `int()` function in Python. On the other hand, Python 3 uses `input()` function which automatically interpreted the type of input entered by the user. However, we can cast this value to any type by using primitive functions (`int()`, `str()`, etc.)

1. In Python 2, the implicit string type is ASCII, whereas, in Python 3, the implicit string type is Unicode.
2. Python 3 doesn't contain the xrange() function of Python 2. The xrange() is the variant of range() function which returns a xrange object that works similar to Java iterator. The range() returns a list for example the function range(0,3) contains 0, 1, 2.

History of Python 2

- Python 2.0 - October 16, 2000
- Python 2.1 - April 17, 2001
- Python 2.2 - December 21, 2001
- Python 2.3 - July 29, 2003
- Python 2.4 - November 30, 2004
- Python 2.5 - September 19, 2006
- Python 2.6 - October 1, 2008
- Python 2.7-July 3, 2010

History of Python 3

- Python 3.0 - December 3, 2008
- Python 3.1 - June 27, 2009
- Python 3.2 - February 20, 2011
- Python 3.3 - September 29, 2012
- Python 3.4-March 16, 2014
- Python 3.5 - September 13, 2015
- Python 3.6- October 2016
- Python 3.7- June 2018.

Basis of comparison	Python 3	Python 2
Release Date	2008	2000
Function print	<code>print ("hello")</code>	<code>print "hello"</code>
Division of Integers	Whenever two integers are divided, you get a float value	When two integers are divided, you always provide integer value.
Unicode	In Python 3, default storing of strings is Unicode.	To store Unicode string value, you require to define them with "u".
Syntax	The syntax is simpler and easily understandable.	The syntax of Python 2 was comparatively difficult to understand.
© Ali Asgar Hussain © Sardar Sardar Khan		

Rules of ordering Comparisons	In this version, Rules of ordering comparisons have been simplified.	Rules of ordering comparison are very complex.
Iteration	The new Range() function introduced to perform iterations.	In Python 2, the xrange() is used for iterations.
Exceptions	It should be enclosed in parenthesis.	It should be enclosed in notations.
Leak of variables	The value of variables never changes.	The value of the global variable will change while using it inside for-loop.
Backward compatibility	Not difficult to port python 2 to python 3 but it is never reliable.	Python version 3 is not backwardly compatible with Python 2.
Library	Many recent developers are creating libraries which you can only use with Python 3.	Many older libraries created for Python 2 is not forward-compatible.
© Aif Alagaddasain Khan		

Python 2 vs. Python 3 Example Code

- ▶ Print “hello world”
- ▶ Print(“Hello World”)

Prime reasons for using Python 3.x versions:

- Python 3 supports modern techniques like AI, machine learning, and data science
- Python 3 is supported by a large Python developer's community. Getting support is easy.
- Its easier to learn Python language compared to earlier versions.
- Offers Powerful toolkit and libraries
- Mixable with other languages

Features of Python

- ▶ **Simple:- Python = Simplicity** We cannot stress this point enough, but Python is not only easy to learn but also easy to use and implement. With a syntax similar to English, you can master the nitty-gritty of Python coding in a few days. Moreover, Python is dynamically-typed, which makes indentation mandatory, thereby enhancing its readability factor.
- ▶ **It is an open-source language:-** You don't need to pay charges to install and use Python – it is open-source. What this means is that the source code of Python is freely available to the public. You can download it from Python's official [website](#). Not only that, Python supports the **FLOSS (Free/Libre and Open Source Software)** model, which means you can also change it and distribute it. This allows the Python community to tweak it and improve its features continuously.

- ▶ **Dynamically Typed:-** In Python we are not required to declare datatype for variables. Whenever we are assigning the value, based on value, type will be allocated automatically. Hence Python is considered as dynamically typed language.
- ▶ But Java, C etc are Statically Typed Languages because we have to provide datatype at the beginning only.
- ▶ This dynamically typed nature will provide more flexibility to the programmer.

- ▶ **It is a high-level language:-** Since Python is a high-level language, you need not remember its system architecture, not do you need to perform memory management. This feature contributes to Python's user-friendliness.
- ▶ **It is interpreted:-** Unlike compiled languages like C++ and Java wherein you must compile the code and then run it, Python is an interpreted language. What this means is that instead of executing the source code all at once, Python executes it line by line. This makes it easier to debug a Python code because you can do it while writing the code.

- ▶ **It is both object-oriented and functional:-** An object-oriented programming language is one that can model real-world data, while a functional language focuses on functions (code that can be reused). Python supports both object-oriented and functional programming features. Also, unlike Java, Python supports multiple inheritances.
- ▶ **It is portable:-** Python is portable and highly flexible, meaning, a Python code written for a Windows machine or a Linux machine can also run on iOS, and vice versa – you don't need to make any alterations in the code. So, with Python eliminates the need to write different code for different machines (just make sure there's no system-dependent feature in your Python code).

- ▶ **It is extensible and embeddable:-** Python is an extensible language, as it allows you to write specific parts of your Python code in other programming languages such as C++. Similarly, you can also embed your Python code in the source code of other languages. This allows you to integrate Python's scripting functionalities into a code written in another language.
- ▶ **It comes with a vast collection of libraries:-** When you download Python, you will automatically download the extensive collection of Python libraries with it. These libraries are built-in, so you don't have to write individual code for every single thing. Python has libraries and packages for web browsers, threading, databases, regular expressions, image manipulation, documentation-generation, unit-testing, CGI, email, and much more.

- ▶ **Platform Independent** :- Once we write a Python program, it can run on platform without rewriting once again. Internally PVM is responsible to convert into machine understand form

Applications of Python

- ▶ **Web Development:-** When it comes to web development, Python should be your go-to tool. Why?
- ▶ That's because Python offers numerous options for web development. For instance, you have Django, Pyramid, Flask, and Bottle for developing web frameworks and even advanced content management systems like Plone and Django CMS. These web frameworks are packed with standard libraries and modules which simplify tasks like content management, database interaction, and interfacing with internet protocols like HTTP, SMTP, XML, JSON, FTP, IMAP, and POP.
- ▶ [Python web frameworks](#) are known for their security, scalability, and flexibility. To add to that, Python's Package Index comes with useful libraries like Requests, BeautifulSoup, Paramiko, Feedparser, and Twisted Python.

- ▶ **Game Development:-** As we mentioned earlier, Python comes loaded with many useful extensions (libraries) that come in handy for the development of interactive games. For instance, libraries like PySoy (a 3D game engine that supports Python 3) and PyGame are two Python-based **libraries used widely for game development**. Python is the foundation for popular games like Battlefield 2, Frets on Fire, World of Tanks, Disney's Toontown Online, Vega Strike, and Civilization-IV.
- ▶ Apart from game development, game designers can also use Python for developing tools to simplify specific actions such as level design or dialog tree creation, and even use those tools to export those tasks in formats that can be used by the primary game engine. Also, Python is used as a scripting language by many game engines.

- ▶ **Scientific and Numeric Applications:-** Python has become a crucial tool in scientific and numeric computing. In fact, Python provides the skeleton for applications that deal with computation and scientific data processing. Apps like FreeCAD (3D modeling software) and Abaqus (finite element method software) are coded in Python.
- ▶ Some of the most useful Python packages for scientific and numeric computation include:
 - SciPy (scientific numeric library)
 - Pandas (data analytics library)
 - IPython (command shell)
 - Numeric Python (fundamental numeric package)
 - Natural Language Toolkit (Mathematical And text analysis)

- ▶ **Artificial Intelligence and Machine Learning :-** AI and ML models and projects are inherently different from traditional software models. When we talk about AI/ML projects, the tools and technologies used and the skillset required is totally different from those used in the development of conventional software projects. AI/ML applications require a language that is stable, secure, flexible, and is equipped with tools that can handle the various unique requirements of such projects. Python has all these qualities, and hence, it has become one of the most favored languages of Data Science professionals.
- ▶ Python's simplicity, consistency, platform independence, great collection of resourceful libraries, and an active community make it the perfect tool for developing AI and ML applications. Some of the best Python packages for AI and ML are:
 - SciPy for advanced computing
 - Pandas for general-purpose data analysis
 - Seaborn for data visualization
 - Keras, TensorFlow, and Scikit-learn for ML
 - NumPy for high-performance scientific computing and data analysis
- ▶ Apart from these libraries, there are also other [Python-based libraries](#) like NLTK, Caffee, PyTorch, and Accord.NET, that are useful for AI and ML projects.

- ▶ **Desktop GUI:-** Python not only boasts of an English-like syntax, but it also features a modular architecture and the ability to work on multiple operating systems. These aspects, combined with its rich text processing tools, make Python an excellent choice for developing desktop-based GUI applications.
- ▶ Python offers many GUI toolkits and frameworks that make desktop application development a breeze. PyQt, PyGtk, Kivy, Tkinter, WxPython, PyGUI, and PySide are some of the best Python-based GUI frameworks that allow developers to create highly functional Graphical User Interfaces (GUIs).

- ▶ **Web Scraping Applications:-** Python is a nifty tool for extracting voluminous amounts of data from websites and web pages. The pulled data is generally used in different real-world processes, including job listings, price comparison, R&D, etc.
- ▶ BeautifulSoup, MechanicalSoup, Scrapy, [LXML](#), Python Requests, Selenium, and Urllib are some of the best Python-based web scraping tools.

- ▶ **Image Processing and Graphic Design Applications:-** Alongside all the uses mentioned above, Python also finds a unique use case in image processing and graphic design applications. The programming language is used globally to design and build 2D imaging software like Inkscape, GIMP, Paint Shop Pro, and Scribus. Also, Python is used in several 3D animation packages such as Blender, Houdini, 3ds Max, Maya, Cinema 4D, and Lightwave, to name a few.

Python First Program

▶ `print("Hello World")`

Python Popular Frameworks and Libraries

- ▶ Python has wide range of libraries and frameworks widely used in various fields such as machine learning, artificial intelligence, web applications, etc. We define some popular frameworks and libraries of Python as follows.
- **Web development (Server-side)** - Django Flask, Pyramid, CherryPy
- **GUIs based applications** - Tk, PyGTK, PyQt, PyJs, etc.
- **Machine Learning** - TensorFlow, PyTorch, **Scikit-learn**, Matplotlib, Scipy, etc.
- **Mathematics** - Numpy, Pandas, etc

Flavors of Python

- ▶ Types of Python compilers are referred as flavors of Python. They help to integrate various types of programming languages in Python some of them are
- ▶ **1. CPython:**
- ▶ It is a Python compiler that was implemented in C language. Even C++ code can be execute using CPython.
- ▶ **2. Jpython or jython:**
- ▶ It is enables Python implementation to be run on Java platform. It runs on JVM.

▶ **3. IronPython:**

- ▶ It is compiler designed for .NET framework, but it is written in C#. It can run on CLR(Common Language Run time).

▶ **4. PyPy:**

- ▶ It is a Python implemented by using Python language itself. It runs fast since JIT is incorporated to PVM.

- ▶ **5. Ruby Python:**

- ▶ It acts as a bridge from Ruby to Python interpreter. It embeds the Python interpreter inside the Ruby application

- ▶ **6. Pythonxy:**

- ▶ It is written in the form of Python(X,Y). It is designed by adding scientific and engineering related packages.

▶ **7. Anaconda Python:**

- ▶ The name Anaconda Python is obtained after redeveloping it to handle large scale data processing, predictive analytics and scientific computing. It handles huge amount of data.

▶ **8. Stackless Python:**

- ▶ Tasklets are the small tasks that are run independently. The communication is done with each by using channels. They schedule, control and suspend the tasklets. Hundreds of tasklets can run by a thread. The thread and tasklets can be created in stackless python. It is a re-implementation of python.

Frozen Binary

- ▶ Frozen binary executables are packages that combine your program's byte code and the Python interpreter into a single executable program. With these, programs can be launched in the same ways that you would launch any other executable program (icon clicks, command lines, etc.). While this option works well for delivery of products, it is not really intended for use during program development. You normally freeze just before shipping, and after development is finished.

Python print() Function

- ▶ The **print()** function displays the given object to the standard output device (screen) or to the text stream file.
- ▶ Unlike the other programming languages, Python **print()** function is most unique and versatile function.

- ▶ The syntax of **print()** function is given below.

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

- ▶ Let's explain its parameters one by one.
- **objects** - An object is nothing but a statement that to be printed. The * sign represents that there can be multiple statements.
- **sep** - The **sep** parameter separates the print values. Default values is ' '.
- **end** - The **end** is printed at last in the statement.
- **file** - It must be an object with a write(string) method.
- **flush** - The stream or file is forcibly flushed if it is true. By default, its value is false.

Comments in Python

- ▶ Comments are descriptions that help programmers better understand the intent and functionality of the program.
- ▶ They are completely ignored by the Python interpreter.

Advantages of Using Comments

- ▶ Using comments in programs makes our code more understandable. It makes the program more readable which helps us remember why certain blocks of code were written.
- ▶ Other than that, comments can also be used to ignore some code while testing other blocks of code. This offers a simple way to prevent the execution of some lines or write a quick pseudo-code for the program.

Single-Line Comments in Python

- ▶ In Python, we use the hash symbol `#` to write a single-line comment.

Multiline Comment

- ▶ Python doesn't offer a separate way to write multiline comments. However, there are other ways to get around this issue.
- ▶ We can use `#` at the beginning of each line of comment on multiple lines.
- ▶ we can use multiline strings (triple quotes) to write multiline comments.
- ▶ The quotation character can either be `'` or `"`.

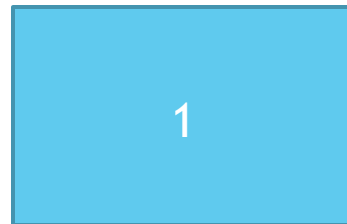
- ▶ Python supports only single line comments.
- ▶ Multiline comments are not available in python. The triple double quotes are actually not multiline comments but they are regular string with the exception that they can span multiple lines. That means memory will be allocated to these string internally. If these string are not assigned to any variable then they are removed from memory by the garbage collector and hence these can be used as comments

Docstrings

- ▶ If we write string inside “""" or ‘’ and if these strings are written as first statements in a module, function, class or a method, then these strings are called documentation strings or docstrings.
- ▶ These docstrings are useful to create an API documentation file from a python program
- ▶ An API documentation file is a text file or html file that contains description of all the features of software, language or a product.
- ▶ When a new software is created , it is duty of the developer to describe all the classes, modules, functions, etc. which are written in that software so that the user will be able to understand the software and use it in a proper way.
- ▶ These description are provided in a separate file either as a text file or an html file. So, we can understand that the API documentation file is like a help file for end user.

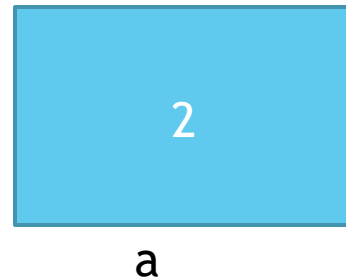
How Python Sees Variables

- ▶ The concept of variable is connected to memory location, in all languages, a variable is imagined as a storage box which can store some value
- ▶ `Int a=1`
- ▶ Memory is allocated with the name 'a' and there the value '1' is stored

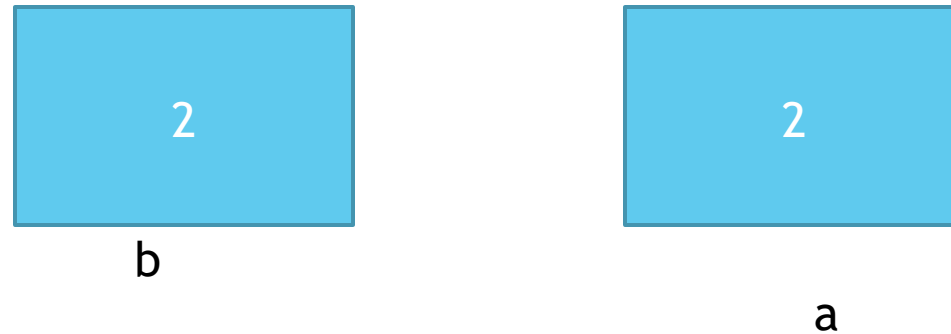


a

- ▶ For every variable we create, there will be new box will be created with the variable name to store the value. If we change the value of the variable, then the box will be updated with the new value
- ▶ `a=2`
- ▶ The new value '2' is stored into the same box



- ▶ When we assign one variable to another variable
- ▶ $b=a$
- ▶ A new memory box will be created by the name 'b' and the value of the variable 'a' is copied into that box



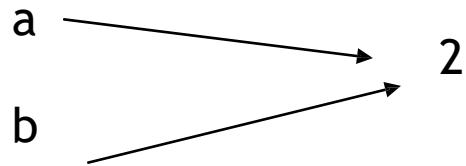
- ▶ In Python, a variable seen as a tag(or name) that is tied to some value
- ▶ `a=1`
- ▶ Means the value '1' is created first in memory and then a tag by the name 'a' is created
- ▶ Python considered the value 1 as object

a $\longrightarrow$ 1

- ▶ If we change the value of 'a'
- ▶ a=2
- ▶ The tag is simple changed to the new value (or object). So value '1' become unreferenced object, it is removed by garbage collector

a $\longrightarrow$ 2 1

- ▶ Assigning one variable to another makes a new tag bound to the same value
- ▶ `b=a`
- ▶ We are storing 'a' value into 'b'. A new tag 'b' will be created that refer to '2'



Datatypes in Python

- ▶ A datatype represents the type of data stored into a variable or memory.
- ▶ The datatypes which are already available in python language are called Built-in datatypes. The datatypes which can be created by the programmers are called user-defined datatypes.

Built-in datatypes

- ▶ None type
- ▶ Numeric types
- ▶ Sequences
- ▶ Sets
- ▶ Mappings

- ▶ None Type:- In python, the 'None' datatype represents an object that does not contain any value. In languages like java it is called 'null' object But in python it is called 'None' object.
- ▶ Use of 'None' is that it is used inside a function as a default value of the arguments. When calling the function. If no value is passed, then the default value will be taken as 'None'. If some value is passed to the function then the value is used by the function
- ▶ In Boolean Expression 'None' datatype represent 'False'.

Numeric Types

- ▶ The numeric type represent numbers. There are three sub types
- ▶ Int
- ▶ Float
- ▶ complex

- ▶ Int datatype:- The int datatype represent an integer number. An integer number is a number without any decimal point or fraction part. For example 13,-5 etc are treated as integer numbers
- ▶ `a=20`
- ▶ Here, 'a' is called int type variable since it is storing 20 which is an integer value. In python there is no limit for the size of an int datatype. It can store large integer numbers conveniently

- ▶ Float datatype:- The float datatype represents floating point numbers. A floating point number is a number that contains a decimal point. For example 0.5, -9.639 are called floating point numbers
- ▶ `a=66.32`
- ▶ 'a' is float type variable since it is storing floating point value. Floating point numbers can also be written in scientific notation where we 'e' or 'E' to represent the power of 10. Here 'e' or 'E' represent exponentiation for example $3.6 * 10^5$ is written as 3.6E5 such number are also treated as floating point numbers
- ▶ `b=3.6e5`
- ▶ The float value $3.6 * 10^5$ is stored into the variable 'b'.
- ▶ The convenience in scientific notation is that it is possible to represent very big numbers using less memory

- ▶ Complex Datatype:- A complex number is a number that is written in the form of $a+bj$ or $a+bJ$
- ▶ Here 'a' represent the real part and 'b' represent the imaginary part of the number. The suffix 'j' or 'J' after b indicates the square root value of -1. the part 'a' or 'b' may contain integer or floats for example $3+5j$, $3.2+3.5j$
- ▶ $a=3+6.9j$

Representing Binary, Octal and Hexadecimal Number

- ▶ A Binary number should be written by prefixing 0b(zero and b) or 0B(zero and B) before the value
- ▶ For example 0b10101,0B101101 are treated as binary numbers.
- ▶ Hexadecimal numbers are written by prefixing 0x(zero and x) or 0X(zero and big X) before the value as 0xA180 or 0X11fb91 etc.
- ▶ Octal numbers are indicated by prefixing 0o(zero and small 0) or 0O(zero and big O) before the actual value as 0o145, 0O773

Converting the Datatypes Explicitly

- ▶ Convert one data type into another type is called type conversion or coercion.
- ▶ `Int(x)` is used to convert the number `x` into `int` type
- ▶ `Float(x)` is used to convert `x` into `float` type
- ▶ `Complex(x)` is used to convert `x` into a complex number with real part `x` and imaginary part zero
- ▶ `Complex(x,y)` is used to convert `x` and `y` into a complex number such that `x` will be the real part and `y` will be the imaginary part

- ▶ `bin(x)` is used to convert `x` into binary number system
- ▶ `oct(x)` is used to convert `x` into octal number system
- ▶ `hex(x)` is used to convert `x` into hexadecimal number system

Bool Datatype

- ▶ The bool datatype in python represents Boolean value. There are only two Boolean values True or False that can be represented by this datatype.
- ▶ Python internally represents True as 1 and False as 0
- ▶ A blank string like "" is also represented as False

Sequences in python

- ▶ A Sequence represents a group of elements or items. For example, a group of integer numbers will form a sequence. There are six type of sequences in python:
- ▶ Str
- ▶ Bytes
- ▶ Bytearray
- ▶ List
- ▶ Tuple
- ▶ Range

- ▶ Str :- In python, str represents string datatypes. A string is represented by a group of characters. Strings are enclosed in single quotes or double quotes. Both are valid
- ▶ `str1="hello world"`
- ▶ We can also write string inside `"""`(triple double quote) or `'`' (triple single quotes) to span a group of lines including spaces.
- ▶ `Str2 = """Computer Programming-III
Python Programming"""`
- ▶ `Str2 = 'Computer Programming-III
Python Programming'`

- ▶ The triple double quotes or triple single quotes are useful to embed a string inside another string as shown below:
- ▶ `Str3="Advance 'python' programming"`

- ▶ The slice operator represents square brackets [and] to retrieve pieces of a string. For example, the characters in a string are counted from 0 onwards. Hence, `str[0]` indicates the 0th character or beginning character in the string.
- ▶ `S[0]` # display 0th character
- ▶ `S[3:7]` # display from 3rd to 6th characters
- ▶ `S[11:]` # display from 11th characters onwards till end
- ▶ `S[-1]` # display first character from the end

- ▶ Repetition operator:- ‘*’ symbol is used to repeat the string for several times, for example `s*n` repeats the string for n times

Bytes Datatype

- ▶ The bytes datatype represents a group of byte numbers just like an array does. A byte number is any positive integer from 0 to 255(inclusive). Bytes array can store numbers in the range from 0 to 255 and it cannot even store negative numbers.
- ▶ We cannot modify or edit any elements in the bytes type array.
- ▶ `x=[1,2,3,4,5]`
- ▶ `x=bytes(x)`

Bytearray Datatype

- ▶ The bytearray datatype array is similar to bytes datatype. The difference is that the bytes type array cannot be modified but the bytearray type array can be modified.
- ▶ It means any element or all the elements of the bytearray type can be modified. To create a bytearray type array, we can use the function bytearray as:
- ▶ `x=[1,2,3,4,5]`
- ▶ `x=bytearray(x)`

List Datatype

- ▶ List in python are similar to arrays in C or java.
- ▶ A List represents a group of elements
- ▶ The main difference between a list and an array is that a list can store different types of elements but an array can store only one type of elements.
- ▶ List can grow dynamically but size of arrays is fixed and they cannot grow at runtime
- ▶ List are represented using square brackets [] and the elements are written in [], separated by commas
- ▶ For example , `list=[1,2,2.3,"medi"]`

Tuple Datatype

- ▶ A tuple is similar to list. A tuple contain a groups of elements which can be of different types.
- ▶ The elements in the tuple are separated by commas and enclosed in parentheses ().
- ▶ The list element can be modified but it is not possible to modify the tuple elements. That means tuple can be created as a read-only list
- ▶ For example:- `tpl=(1,2,3,"medi")`

Range Datatype

- ▶ The range datatype represents a sequence of numbers. The numbers in the range are not modifiable
- ▶ Generally, range is used for repeating a for loop for a specific number of times.
- ▶ To create a range of numbers we can simply write:
- ▶ `R=range(10)`
- ▶ Syntax: `range(start, end, step)`

Sets

- ▶ A Set is an unordered collection of elements much like a set in mathematics. The order of elements is not maintained in the sets.
- ▶ It means the elements may not appear in the same order as they are entered into the set.
- ▶ A set does not accept duplicate elements
- ▶ There are two sub types in sets:
 - ▶ Set datatype
 - ▶ Frozen datatype

Set datatype

- ▶ To create a set , we should enter the elements separated by commas inside curly braces { }.
- ▶ $S=\{1,2,3,5,6\}$
- ▶ Sets are unordered, we cannot retrieve the elements using indexing or slicing operation

Frozen set Datatype

- ▶ The frozen set datatype is same as the set datatype. The main difference is that the elements in the set datatype can be modified; whereas, the elements of frozen set cannot be modified. We can create a frozen set by passing a set to frozen set() function as:
- ▶ For example:
- ▶ $S=\{1,2,3,4,5\}$
- ▶ `Fs=frozenset(S)`

Mapping Types

- ▶ A map represents a group of elements in the form of key value pairs so that when the key is given, we can retrieve the value associated with it.
- ▶ The dict datatype is an example for a map.
- ▶ The 'dict' represents a 'dictionary' that contains pairs of elements such that the first elements represents the key and the next one becomes its value.
- ▶ The key and its value should be separated by a colon (:) and every pair should be separated by a comma.
- ▶ All the elements should be enclosed inside curly brackets { } .

- ▶ We can create a dictionary by typing the roll numbers and the names of students.
- ▶ Hence, roll no. are keys and names will become values.
- ▶ We write these key value pairs inside curly braces as:
- ▶ `D={10:"Rohit", 20:"Amit"}`

Literals in Python

- ▶ A literal is a constant value that is stored into a variable in a program.
- ▶ `a= 10`
- ▶ Here, 'a' is the variable into which the constant value "10" is stored
- ▶ Hence, the value 10 is called literal
- ▶ Since 10 indicates integer value, it is called integer literal

Different type of literals

- ▶ Numeric literals
- ▶ Boolean literals
- ▶ String literals

Numeric Literals

Examples	Literal name
250, -36	Integer Literal
5.562, 3.25e5	Float Literal
0x6A1A	Hexadecimal Literal
00562	Octal Literal
0B101	Binary Literal
3+2J	Complex Literal

Boolean Literal

- ▶ Boolean Literals are the True or False values stored into a bool type variable

String Literals

- ▶ A group of characters is called a string literal.
- ▶ These string literals are enclosed in single quotes (') or double quotes (") or triple quotes (""").

Important Escape Characters in Strings

Escape Character	Meaning
\	New line continuation
\\	Display a single \
\'	Display a single quote
\"	Display a double quote
\b	backspace
\r	Enter
\t	Horizontal tab space
\v	Vertical tab
\n	New line

Determining the Datatype of a variable

- ▶ To know the datatype of a variable or object, we can use the `type()` function.
- ▶ `a= 10`
- ▶ `type(a)`
- ▶ `<class 'int'>`
- ▶ It shows class 'int'. It means the variable 'a' is an object of the class 'int'. It means 'int' is treated as a class. Every datatype is created as an object internally by python.
- ▶ In fact, Every datatype, function, method, class, module, lists, set, etc. are all objects in python

- ▶ Python does not have a char datatype to represent individual characters.

Constants in Python

- ▶ A constant is similar to a variable but its value cannot be modified or changed in the course of the program execution
- ▶ In languages like C and java defining constant is possible. But in python , that is not possible
- ▶ A programmer can indicate a variable as constant by writing its name in all capital letters. For example
- ▶ `PI_VALUE=3.14` is a constant. But its value can be changed

Identifiers

- ▶ An Identifier is a name that is given to a variable or function or class etc. Identifiers can include letters, numbers and the underscore character (_). They should always start with a nonnumeric character. Special symbols such as ?, #, \$, % and @ are not allowed in identifiers.
- ▶ Some examples are name , age etc
- ▶ Note:- Python is a case sensitive programming language. It means capital letters and small letters are identified separately by python

This symbol stores right side literal into left side variable. Thus it is performing an operation. Hence it is called operator

► Experience=14.5

This is a variable its name 'Experience'. Hence this name 'Experience' is 'identifier'

This is the float number stored into variable. Hence it is called floating point literal

Reserve Words

and	del	from	nonlocal	try	as	elif	global	not
assert	else	if	or	with	break	except	import	pass
class	exec	in	print	false	continue	finally	is	raise
def	for	lambda	return	while	yield	True		

Naming Conventions in Python

- ▶ Python developers made some suggestion to the programmers regarding how to write names in the programs. The rules related to write names of package, modules, classes, variables, etc. are called naming conventions
- ▶ Packages: package name should be written in all lower cases letters, When multiple name are used for a name we should separate them using an underscore(_)
- ▶ Modules: Modules name should be written in all lower case letters
- ▶ Classes: Each word of a class name should be start with a capital letter
- ▶ Global variables:- Global variable names should be all lower case letters
- ▶ Instance variable:- instance variable should be all lower case letter

- ▶ Functions:- Functions name should be all lower case letters
- ▶ Methods:- Method name should be all lower case letters
- ▶ Methods Arguments:- In case of instance method , the first argument should be 'self'. In case of class methods, their first argument should be 'cls'
- ▶ Constants:- Constant name should be written in all capital letters

Operators

- ▶ An Operator is a symbol that performs an operation
- ▶ An operator acts on some variable is called operands
- ▶ For example , if we write $a+b$, the operator '+' is acting on two operand a and b
- ▶ If an operator acts on a single variable is called unary operator
- ▶ If an operator acts on two variable is called binary operator
- ▶ If an operator acts on three variable is called ternary operator

Classify the Operator

- ▶ Arithmetic Operator
- ▶ Assignment Operator
- ▶ Unary Operator
- ▶ Relational Operator
- ▶ Logical Operator
- ▶ Boolean Operator
- ▶ Bitwise Operator
- ▶ Membership operator
- ▶ Identity Operator

Arithmetic Operators

Operator	Meaning	Example
+	Addition Operator. Add two values	$a+b$
-	Subtraction Operator, Subtract one value from another	$a-b$
*	Multiplication Operator	$a*b$
/	Division Operator	a/b
%	Modulus Operator, Gives remainder of all division	$a\%b$
**	Exponent operator, calculate exponential power	$a**b$
//	Integer division. This is called floor division and gives only integer quotient	$a//b$

Using Python Interpreter as Calculator

- ▶ Python Interpreter as a simple calculator that can perform basic arithmetic calculation

Assignment Operator

- ▶ These operators are useful to store the right side value into a left side variable. They can also be used to perform simple arithmetic operation like addition, subtraction etc and then store the result into a variable
- ▶ Let $x=20$, $y=10$, $z=5$

$x=20, y=10, z=5$

Operator	Example	Meaning	Result
= (Assignment Operator)	$z=x+y$	$X+y$ is stored into z	$z=30$
+= (Addition Assignment Operator)	$Z+=x$	$Z=z+x$	$Z=25$
-= (Subtraction assignment Operator)	$Z-=x$	$Z=z-x$	$Z=-15$
= (Multiplication assignment Operator)	$Z=x$	$Z=z*x$	$Z=100$
/= (Division Assignment Operator)	$z/=x$	$Z=z/x$	$Z=0.25$
%= (Modulus assignment Operator)	$Z\%=x$	$Z=z\%x$	$Z=5$
= (Exponential Assignment operator)	$Z=y$	$Z=z**y$	$Z=9765625$
//= (Floor division assignment Operator)	$z//=y$	$Z=z//y$	$Z=0$

Assign Value in Variable

- ▶ Assign same value in two variable in the same statement
- ▶ `a=b=1`
- ▶ Assign the different values in two variables
- ▶ `a=1; b=2`
- ▶ `a , b=1,2`

Unary Minus operator

- ▶ The unary Minus operator is denoted by the symbol minus (-). When this operator is used before a variable, its value is negated. That means if the variable value is positive, it will be converted into negative and vice versa.
- ▶ `n=10`
- ▶ `n= -n`
- ▶ `print(n)`
- ▶ It will be display -10

Relational Operators

- ▶ Relational operators are used to compare two quantities. We can understand whether two values are same or which one is bigger or which one is lesser etc.
- ▶ Using these operators will result in True or False depending on the values compared

a=1,b=2

Operator	Example	Meaning	Result
>	a>b	Greater than operator	False
>=	a>=b	Greater than or equal operator	False
<	a<b	Less than operator	True
<=	a<=b	Less than or equal operator	True
==	a==b	Equal operator	False
!=	a!=b	Not equals operator	True

Logical Operators

- ▶ Logical operators are useful to construct compound conditions. A compound condition is a combination of more than one simple condition. Each of the simple condition is evaluated to True or False and then decision is taken to know whether the total condition is True or False

x=1, y=2

Operator	Example	Meaning	Result
and	x and y	Logical And, If x is false , it returns x, otherwise it return y	2
or	x or y	Logical Or, if x is false, it returns y, otherwise it returns x	1
not	not x	Logical not, True if operand is false	False

Boolean Operators

- ▶ Boolean operators act upon 'bool' type literals and they provide 'bool' type output. It means the result provided by Boolean operators will be again either True or False

x=True, y=False

Operator	Example	Meaning	Result
and	x and y	Boolean and operator, if both x and y are true then it return true otherwise false	False
or	x or y	Boolean or Operator, If either x or y is True , then it returns True, else False	True
not	not x	Boolean not Operator, If x is True it returns False, else True	False

Bitwise Operators

- ▶ These operators act on individual bits (0 and 1) of the operands. We can use bitwise operators directly on binary numbers or on integer also
- ▶ When we use these operators on integers, these numbers are converted into bits(binary number system) and then bitwise operator act upon those bits. The result given by these operators are always in the form of integers.

Types of Bitwise Operators

- ▶ Bitwise Complement operator (~)
- ▶ Bitwise AND operator (&)
- ▶ Bitwise OR Operator (|)
- ▶ Bitwise XOR Operator (^)
- ▶ Bitwise Left Shift Operator (<<)
- ▶ Bitwise Right Shift Operator (>>)

Bitwise Complement Operator (~)

- ▶ This operator gives a complement of a given number.
- ▶ **Returns one's complement of the number.**
- ▶ $a = 10 = 1010$ (Binary)
- ▶ $\sim a = \sim 1010$
- ▶ $= -(1010 + 1)$
- ▶ $= -(1011)$
- ▶ $= -11$ (Decimal)

Bitwise AND Operator (&)

- ▶ Returns 1 if both the bits are 1 else 0.
- ▶ $a = 10 = 1010$ (Binary)
- ▶ $b = 4 = 0100$ (Binary)

- ▶ $a \& b = 1010$
 $\&$
 0100
 $= 0000$
 $= 0$ (Decimal)

X	Y	X&Y
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise OR Operator (|)

- ▶ Returns 1 if either of the bit is 1 else 0.
- ▶ $a = 10 = 1010$ (Binary)
- ▶ $b = 4 = 0100$ (Binary)

- ▶ $a | b = 1010$
 |
 0100
 = 1110
 = 14 (Decimal)

X	Y	X Y
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise XOR operator (^)

- ▶ Returns 1 if one of the bit is 1 and other is 0 else returns false.
- ▶ $a = 10 = 1010$ (Binary)
- ▶ $b = 4 = 0100$ (Binary)

- ▶ $a \& b = 1010$
- ▶ $\quad \quad \quad \wedge$
- ▶ $\quad \quad \quad 0100$
- ▶ $\quad \quad \quad = 1110$
- ▶ $\quad \quad \quad = 14$ (Decimal)

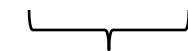
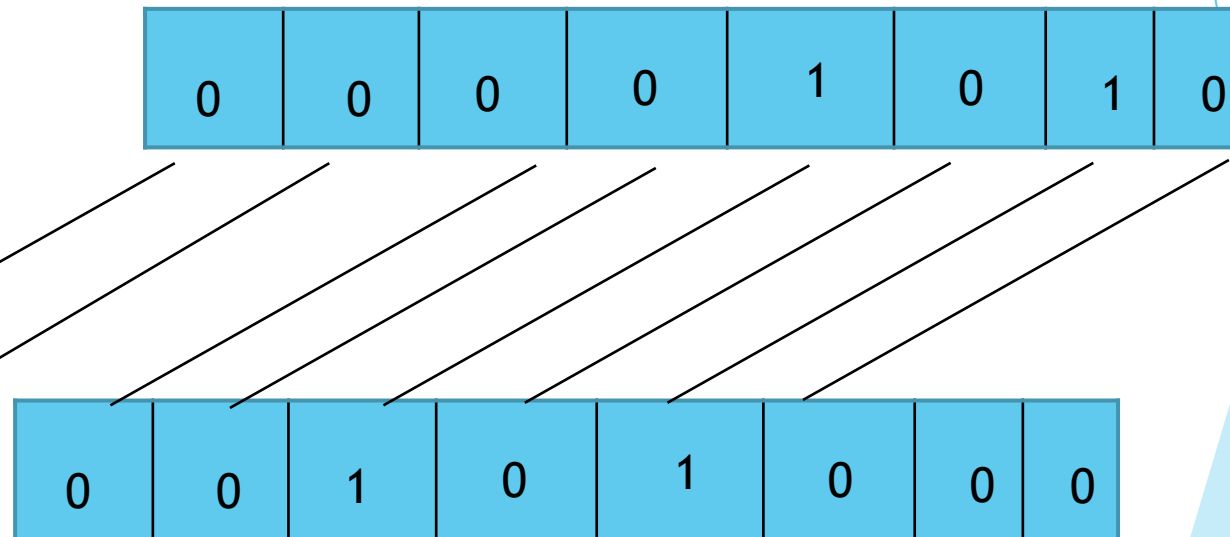
X	Y	X^Y
0	0	0
0	1	1
1	0	1
1	1	0

Bitwise Left Shift Operator (<<)

- ▶ Shifts the bits of the number to the left and fills 0 on voids right as a result
- ▶ This operator shifts the bits of the number toward left a specified number of positions.
- ▶ The Symbol for this operator is <<, read as double less than
- ▶ If we write $x \ll n$ it means shift the bits of x towards left n positions

► $X=10 = 00001010$

$X \ll 2$

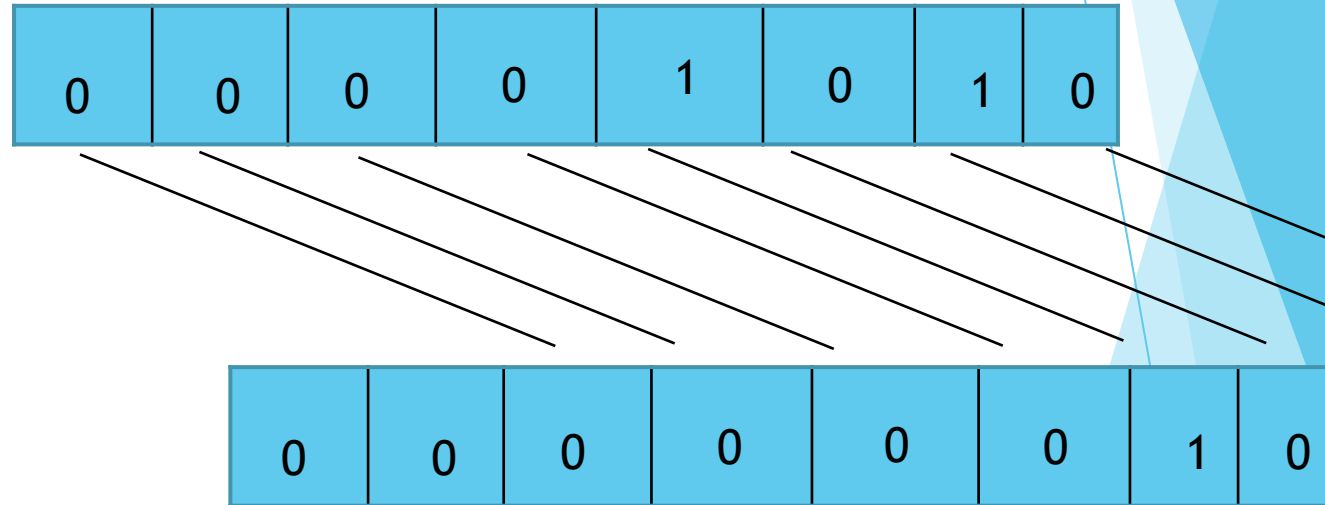


Fill 0's

Bitwise Right Shift Operator

- ▶ Shifts the bits of the number to the right and fills 0 on voids left as a result.
- ▶ $X=10=00001010$

$X \gg 2$



Fill 0's

Membership Operators

- ▶ The membership operators are useful to test for membership in a sequence such as string, list, tuples or dictionaries.

Types of Membership Operator

- ▶ In
- ▶ Not in

In operator

- ▶ This operator returns true if an elements found in the specified sequence. If the element is not found in the sequence then it returns false

Not in operator

- ▶ This works in reverse manner for 'in' operator. This operator returns true if an element is not found in the sequence. If the element found then it returns false

Identity Operators

- ▶ These operators compare the memory location of two objects. Hence it is possible to know whether the two objects are same or not. The memory location of an object can be seen using the `id()` function. This function returns an integer number, called the identity number that internally represent the memory location of an object.

Types of identity operators

- ▶ Is
- ▶ Is not

Is operator

- ▶ The is operator is useful to compare whether two objects are same or not. It will internally compares the identity number of the objects. If the identity numbers of the objects are same, it will return true, otherwise it return false

Is not operator

- ▶ This is not operator returns True, If the identity numbers of two objects being compared are not same. If they are same it will return False

Operator Precedence

Operators	Meaning
()	Parentheses
**	Exponent
+X, -X, ~X	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Input and Output

- ▶ To display output or results , Python provides the `print()` function. This function can be used in different formats.
- ▶ `Print()`:- When the `print()` function is called simply, it will throw the cursor to the next line it means that a blank line will be displayed.
- ▶ `Print("string")`:- A string represents a group of characters. When a string is passed to the `print()` function the string is displayed as it is
- ▶ The operator `'+'` can be used to join one string with another string. Hence `'+'` is called concatenation operator when used on strings

- ▶ Print(variable list):- we can display the values of variables using the print() function. A list of variables can be supplied to the print() function
- ▶ a=5;b=6
- ▶ Print(a,b)
- ▶ The values in the output are separated by a space by default.
- ▶ To separate the output with comma we should use 'sep' attribute.
- ▶ 'sep' attribute represents separator
- ▶ Sep='characters
- ▶ Print(a,b,sep=",")

- ▶ `print()` function throw the cursor into the next line after displaying the output
- ▶ We want next output in the same line.
- ▶ “end” which indicates the ending characters for the line and it will display the next output in the same line
- ▶ `Print(“hello”, end=“”)`
- ▶ If we want to display the output in the same line with tab space
- ▶ `Print(“hello”, end=“\t”)`
- ▶ `Print(“world”)`

- ▶ `Print(object)`: we can pass objects like lists, tuples or dictionaries to the `print()` function to display the elements of those objects
- ▶ `Print("string", variable list)`: The most common use of the `print()` function is to use strings along with variables inside the `print()` function
- ▶ `Print(formatted string)`: The output displayed by the `print()` function can be formatted. The operator `'%'` can be used for this purpose. It joins a string with a variable or value in the following format:
 - ▶ `Print("formatted string" %(variable list))`
 - ▶ `X=5`
 - ▶ `Y=6`
 - ▶ `Print("value of x=%i and y=%i" %(x,y))`

- ▶ N="rohit"
- ▶ print("name=%s" % n)
- ▶ Print("hello %20s" %n): it will allot 20 space and string is displayed right aligned
- ▶ Print("hello %-20s" %n): it will allot 20 space and string is displayed left aligned
- ▶ print("hello (%-20s)"%n[0:2])

- ▶ To display floating point values, we can use %f in the formatted string. Of we use %5.2f then the float value is displayed in 5 spaces and within these spaces, a decimal point and next 2 fraction digits.
- ▶ A=123.569
- ▶ Print("The value is %5.2f"%a)

- ▶ Inside the formatted string , we can use replacement field which is denoted by a pair of curly braces { }.
- ▶ We can mention names or indexes in these replacement fields. These names or indexes represent the order of the values. After formatted string , we should write member operator and then format() method where we should mention the values to be displayed
- ▶ `Print("format string with replacement fields".format(values))`
- ▶ `a=10;b=20;c=30`
- ▶ `Print("value of a={0}".format(a))`
- ▶ `Print("value of a={0} and b={1}".format(a,b))`
- ▶ `Print("value of a={} and b={}".format(a,b))`
- ▶ `Print("value of a={one} and b={two}".format(one=a,two=b))`
- ▶ `print("value of a={:.2f} and b={:.2f}".format(a,b))`

Input Statements

- ▶ To accept input from keyboard, python provides the `input()` function. This function takes a value from the keyboard and return it as a string.
- ▶ `Str=input()`
- ▶ `Str=input("Enter your name")`
- ▶ `A=int(input("Enter a num"))`
- ▶ `B=float(input("enter a num"))`

Accept more than one input in the same line

▶ `a,b=[int(x) for x in input("Enter number").split()]`

Eval() Function

- ▶ The eval() function takes a string and evaluates the result of the string by taking it as a python expression
- ▶ For example. “a+b+6”
- ▶ If we pass the string to the eval() function it will evaluate the string and return the result

Command Line Argument

- ▶ We can design our program in such a way that we can pass inputs to the program at the time of running the program at command prompt
- ▶ Running program when we passing the arguments are called command line arguments, so command line arguments are the value passed to a python program at command prompt. Command line argument are passed to the program from outside the program.
- ▶ All argument should be entered from the keyboard separated them by a space
- ▶ These arguments are stored by default in the form of strings in a list with the name “argv” which is available in sys module
- ▶ “argv” is a list that contain all the values passed to the program
- ▶ Argv[0] represent the name of the program, argv[1] represent the first value and argv[2] represent the second value and so on

Add.py	5	6
argv[0]	argv[1]	argv[2]

Sequential Execution

- ▶ The statements in the program are normally executed one by one. This type of execution is called sequential execution

Control Statements

- ▶ Control statements are statements which control or change the flow of execution. The following are the control statements available in python
- ▶ If statements
- ▶ If else statements
- ▶ If elif else statements
- ▶ While loop
- ▶ For loop
- ▶ Else suite
- ▶ Break
- ▶ Continue
- ▶ Pass
- ▶ Assert
- ▶ Return
- ▶ Switch statement is not available in python

Indentation

- ▶ Indentation is very important in python. It refers to space that are used in the beginning of a statement. The statement with same indentation belong to same group called a suite.
- ▶ By default python uses 4 spaces but it can be increased or decreased by the programmers

If statement

- ▶ This statement is used to execute one or more statement depending on whether a condition is true or not. The syntax or correct format of if statement is
- ▶ If condition:
- ▶ statements

If-else statement

- ▶ If else statements executes a group of statements when a condition is True; otherwise it will execute another group of statements
- ▶ The syntax of if-else statement is:
- ▶ if condition:
- ▶ statements
- ▶ else:
- ▶ statements

If-elif-else statement

- ▶ Sometimes, the programmer has to test multiple conditions and execute statement depending on those conditions. If-elif-else statement is useful in such situation
- ▶ Syntax:
- ▶ If condition1:
- ▶ statements
- ▶ elif condition2:
- ▶ statements
- ▶ elif condition3:
- ▶ statements
- ▶ elif condition4:
- ▶ statements
- ▶ else:
- ▶ statements

While loop

- ▶ A statement is executed only once from top to bottom but A loop is useful to execute a statement repeatedly.
- ▶ While and for are loops in python. They are useful to execute a group of statements repeatedly several times.
- ▶ Syntax of while loop:
- ▶ While condition:
 - ▶ statements
- ▶ Statements represent one statements or group of statements

For loop

- ▶ For loop is useful to iterate over the elements of a sequence. It means the for loop can be used to execute a group of statements repeatedly depending upon the number of elements in the sequence.
- ▶ The for loop can work with sequence like string, list, tuple, range etc.
- ▶ Syntax:
- ▶ For var in sequence:
- ▶ statements

Infinite Loop

- ▶ Loop will be execute forever
- ▶ Infinite loops are drawback in a program because when the user is caught in an infinite loop he cannot understood how to come out of the loop. So it is always recommended to avoid infinite loop in any program

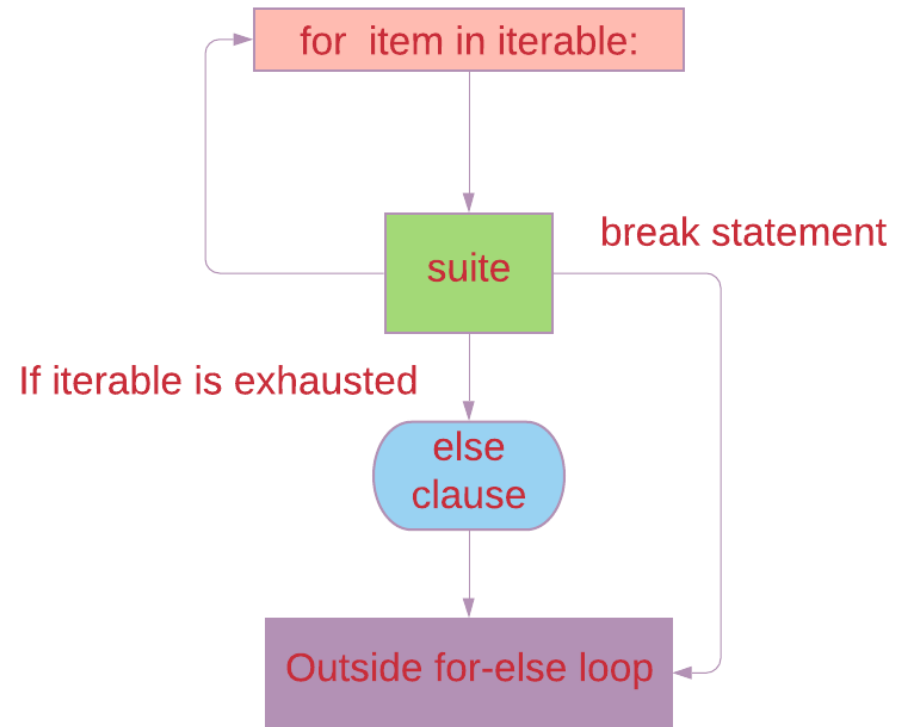
Nested Loop

- ▶ It is possible to write one loop inside another loop such loop is called nested loop
- ▶ We can write a for loop inside a while loop or a for loop inside another for loop
- ▶ For l in range(3):
 - ▶ for j in range(4):
 - ▶ print(l,j)

Else suite

- ▶ To use 'else' statements along with for loop or while loop

For with else	While with else
For var in sequence: statements Else: statements	While condition: statements Else: statements



Break statement

- ▶ The break statement can be used inside a for loop or while loop to come out of the loop.
- ▶ When break is executed the python interpreter jumps out of the loop to process the next statement in the program

Continue statement

- ▶ The continue statement is used in a loop to go back to the beginning of the loop.
- ▶ When continue is executed , the next repetition will start
- ▶ When continue is executed, the subsequent statement in the loop are not executed

Pass statement

- ▶ The pass statement does not do anything. It is used with 'if ' statement or inside a loop to represent no operation. We use pass statement when we need a statement syntactically but we do not want to do any operation.
- ▶ A more meaningful usage of the pass statement is to inform python interpreter not to do anything when we are not interested in the result

Assert statement

- ▶ The assert statement is useful to check if a particular condition is fulfilled or not
- ▶ Syntax
- ▶ Assert expression, message
- ▶ Assert $x > 100$, “value should be less than 100”
- ▶ Python interpreter checks if $x > 100$ is true or not. If it is true then the next statement will execute else it will display `AssertionError` along with the message “value should be less than 100”

Return statement

- ▶ A function represents a group of statement to perform a task. The purpose of a function is to perform some task and in many cases a function returns the result.
- ▶ A function starts with the keyword `def` that represents the definition of the function. After der the function should be written. Then we should write the variable in the parentheses.
- ▶ For example:
- ▶ `Def sum(a,b):`
- ▶ `return a+b`

String and characters

- ▶ A string represents a group of characters. Strings are important because most of the data that we use in daily life will be in the form of strings
- ▶ For example, the names of persons, their addresses, vehicle numbers, their credit car numbers, etc. are all strings
- ▶ In python the str data type represents a string
- ▶ Python handle strings and characters almost in the same manner. There is no separate datatype to represent individual character

Create Strings

- ▶ S1='Hello Mediacaps'
- ▶ S2="Hello mediacaps University"
- ▶ There is no difference between the single quote and double quotes while creating the string. Both will work in the same manner
- ▶ Sometimes we can use triple single quotes or triple double quotes to represent strings.

- ▶ It is possible to use escape character inside the string
- ▶ To nullify the effect of escape character, we can create the string as a “raw” string by adding “r” before the string
- ▶ `S1=r“Welcome to medi-caps\tUniversity\n”`

Unicode Characters

- ▶ To create a string with Unicode characters, we should add 'u' at the beginning of the string. Unicode is a standard to include the alphabet of various human languages into programming languages like python or java.
- ▶ For example, it is possible to display the alphabet of hindi, French and german languages using Unicode system
- ▶ Each Unicode character contains 4 digit preceded by a \u
- ▶ `print(u'\u0915\u094b\u0930 \u092a\u0948\u0925\u0964\u0928')`

Length of a string

- ▶ Length of a string represent the number of characters in a string. To know the length of a string we can use the len() function.
- ▶ Str1="hello"
- ▶ Print(len(str1))

Indexing in strings

- ▶ Index represents the position number. Index is written using square braces[]. By specifying the position number through an index, we can refer to the individual elements of a string.
- ▶ For example `str[0]` refers to the elements of the string and `str[1]` refers to the 2nd element of the string. Thus, `str[i]` can be used to refer to *i*th element of the string. So *i* is called the string index because it is specifying the position number of the element in the string
- ▶ When we use index as a negative number it refers to elements in the reverse order. Thus `str[-1]` refers to the last element and `str[-2]` refers to the second last element

Positive index

0	1	2	3	4	5	6	7	8	9
M	E	D	I	-	C	A	P	S	-

Negative index

-10 -9 -8 -7 -6 -5 -4 -3 -2 -1

Slicing the string

- ▶ A slice represents a part or piece of a string. The format of slicing is:
- ▶ `Stringname[start: stop: stepsize]`
- ▶ If start and stop are not specified then slicing is done from 0th to n-1 elements
- ▶ If step size is not written then it is taken to be 1.

- ▶ `Str[0:9:1]` # access string from 0th to 8th element in steps of 1
- ▶ `Str[0:9:2]` # access string from 0th to 8th element in steps of 2
- ▶ `Str[:,]` #access string from 0th to 8th last character
- ▶ `Str[2:4:1]` #access string from 2th to 3th element in steps of 1
- ▶ `Str[:,2]` # access string from 0th to last element in steps of 2
- ▶ `Str[2::]` # access string from 2nd to ending in steps of 1
- ▶ `Str[:4:]` #access string from 0th to 3th element in steps of 1
- ▶ `Str[-4:-1]` #access string from -4 to -2 from left to right element in steps of 1
- ▶ `Str[-6::]` #access string from -6th to end of the string
- ▶ `Str[-1:-4:-1]` # access string from -1th to -3th element in steps of 1 from right to left
- ▶ `Str[-1::-1]` #access string from -1th to end string element in steps of 1 from right to left

Repeating the Strings

- ▶ The repetition operator is denoted by “*” symbol and is useful to repeat the string for several times.
- ▶ `S=“hello”`
- ▶ `Print(s*2)`
- ▶ `Print(s[2:5]*3)`

Concatenation of Strings

- ▶ We can use '+' on strings to attach a string at the end of another strings. This operator s called "+" operator when used on numbers. But, when used on strings it is called concatenation operator since it joins or concatenates the string

Checking Membership

- ▶ We can check if a string or character is a member of another string or not using “in” or “not in” operators. The “in” operators return true if string is found in the main string. It returns false if the string is not found in the main string

Comparing Strings

- ▶ We can use relational operators like `>`, `>=`, `<`, `<=`, `==` or `!=` operators to compare two strings. They return Boolean value i.e. either True or False depending on the string being compared

Removing Spaces from a String

- ▶ A space is also considered as a character inside a string. Sometimes the unnecessary spaces in a string will lead to wrong results.
- ▶ Rstrip():- `rstrip()` method which is used to remove space from the right side of the string.
- ▶ Lstrip():- `lstrip()` method which is used to remove space from the left side of the string.
- ▶ Strip():- `strip()` method which is used to remove space from both sides

Finding Sub String

- ▶ The find(), rfind(), index(), rindex() method are useful to locate substrings in a string. These method return the location of the first occurrence of the sub string in the main string.
- ▶ The find() and index() methods search for the substring from the beginning of the main string
- ▶ The rfind() and rindex() methods search for the substring from right to left i.e. in backward order.
- ▶ The find() method return -1 if the substring is not found in the main string.
- ▶ The index() method returns “ValueError” exception is the substring is not found
- ▶ Syntax:
- ▶ `Mainstring.find(substring,beginning,ending)`
- ▶ The find() and index() method returns only the first occurrence of the sub string

Count substring

- ▶ The method `count()` is available to count the number of occurrence of a substring in a main string.
- ▶ `Stringname.count(substring)`
- ▶ `Stringname.count(substring,beg,end)`

Immutable Strings

- ▶ An immutable object is an object whose content cannot be changed. On the other hand, a mutable object is an object whose content can be changed as and when required. In python, numbers, strings and tuples are immutable. List, sets , dictionaries are mutable objects.

Reason:

- ▶ Performance:- When an object is immutable, it will have fixed size in memory since it cannot be modified. Because strings are immutable, we can easily allocate memory space for them at creation time and the storage requirements are fixed and unchanging. Hence it takes time to allocate memory for strings are fixed and less time to access them. This increase the performance of the software
- ▶ Security:- Since Strings object are immutable, any attempts to modify the existing string object will create a new object in memory. Thus the identity number of the new object will change lets the programmer to understand that somebody modified the original string.

Replacing a String

- ▶ The replace method is useful to replace a sub string in a string with another sub string
- ▶ Syntax:
- ▶ `Stringname.replace(old,new)`

Changing Case

- ▶ Python offers 4 methods that are useful to change the case of a string. They are `upper()`, `lower()`, `swapcase()`, `title()`.
- ▶ `Upper()`:- The upper method is used to convert all the characters of a string into uppercase or capital letters
- ▶ `Lower()`:- the `lower()` method converts the string into lowercase or into small letters
- ▶ `Swapcase()`:- The `swapcase()` method converts the capital letters into small letters and vice versa
- ▶ `Title()`:- The `title()` method converts the string such that each word in the string will start with a capital letter and remaining will be small letters.

Splitting and Joining

- ▶ The split method is used to break a string into pieces. These pieces are returned as a list.
- ▶ `S="hello world"`
- ▶ `s.split(" ")`
- ▶ `Output=["hello","world"]`

- ▶ Join() method will be used to join the string and make a single string
- ▶ Syntax:- separator.join(str)
- ▶ Print(“ “.join(s))

Checking starting and ending of a string

- ▶ The `startswith()` method is useful to know whether a string is starting with a substring or not
- ▶ `str.startswith(str)`
- ▶ The `endswith()` method is useful to know whether a string is ending with a substring or not

String Testing Methods

Method	Description
Isalnum()	This methods return true if all characters in the string are alphanumeric(AtoZ,a to z, 0to9) and there is atleast one character otherwise False
Isalpha()	Returns True if the string has at least one character and all characters are alphabetic; otherwise False
Isdigit()	Return True if the string contains only numeric digit otherwise False
Islower()	Return true if the string contains at least one letter and all the letter contains are in lower case otherwise false

Sorting String

- ▶ We can sort a group of string into alphabetical order using `sort()` and `sorted()` method

List and Tuples

- ▶ List:- A List is similar to an array that consists of a group of elements or items. Just like an array, a list can store elements But, there is one major difference between an array and list.
- ▶ An array can store only one type of elements where a list can store different types of elements
- ▶ List are the most used datatype in python program
- ▶ `L=[10,20,30,"Rohit","m"]`

Creating list using range() function

- ▶ Range() function to generate a sequence of integers which can be stored in a list
- ▶ `k=list(range(10))`

Concatenation of list

- ▶ We can simple use “+” operator on two list to join them
- ▶ $A=[10,20,30]$
- ▶ $B=[5,6,7]$
- ▶ $A+B$
- ▶ $[10,20,30,5,6,7]$

Repetition of list

- ▶ We can repeat the elements of a list “n” number of times using “*” operator

Membership In List

- ▶ We can check if an element is a member of a list or not using “in” or “not” in operator

Aliasing and cloning

- ▶ Given a new name to an existing list is called aliasing
- ▶ `X=[1,2,3]`
- ▶ `Y=x`
- ▶ Clone:- a sperate copy of all the elements is stored into y
- ▶ `Y=x[:]`

Methods to Process List

Method	Example	Des
Sum()	List.sum()	Sum of all elements
Index()	List.index(x)	Return first occurrence
Append()	List.append(x)	Append x into list
Insert()	List.insert(l,x)	Insert x into list
Copy()	List.copy()	Copy all list element into new list
Extend()	List.extend(list1)	Append list 1 to list
Count()	List.count(x)	Return no. of occurrence of x
Remove()	List.remove(x)	Remove x
Pop()	List.pop()	Remove end element
Sort()	List.sort()	Sort the element

Max() and Min()

- ▶ Return the biggest and smallest elements from a list

Tuples

- ▶ A Tuple is a python sequence which stores a group of elements or items.
- ▶ Tuples are similar to list but the main difference is tuples are immutable whereas list are mutable.
- ▶ Tuple are immutable, once we create a tuple we cannot modify its elements. Hence we cannot perform operation like append, extend, insert, remove, pop and clear on tuples.
- ▶ Tuple are generally used to store data which should not be modified and retrieve that data on demand

Creating tuple

- ▶ `T=()` # empty tuple
- ▶ `T=(5,)` #tuple with one element
- ▶ `T=(5,6,"hello")`
- ▶ `T=(2,6,9)`
- ▶ `T=9,8,6,3` #tuple with no braces

Create tuple from list

- ▶ Convert a list into tuple using tuple() function

Create tuple using range function

▶ `t=tuple(range(10))`

Basic Operations on tuples

- ▶ Length
- ▶ Concatenation
- ▶ Repetition
- ▶ Membership
- ▶ iteration

Functions to process tuples

- ▶ `Len()`
- ▶ `Min()`
- ▶ `Max()`
- ▶ `Count()`
- ▶ `Index()`
- ▶ `Sorted()`

Nested Tuple

- ▶ A tuple inserted inside another tuple is called nested tuple

Operations on Tuple

- ▶ Insert elements in a tuple
- ▶ Modify elements in a tuple
- ▶ Delete elements in a tuple

Dictionaries

- ▶ A dictionary represents a group of elements arranged in the form of key-value pairs
- ▶ A first element is considered as “key” and the immediate next elements is taken as its “value”.
- ▶ The key and value are separated by a colon(:)
- ▶ All the key-value pairs in a dictionary are inserted in curly braces {}

Points to remember

- ▶ Keys should be unique. It means duplicate keys are not allowed. If we enter same key again, the old key will be overwritten and only the new key will be available
- ▶ Key should be immutable type. For example we can use a number, string or tuples as keys since they are immutable. We cannot use list or dictionaries as keys

Methods

- ▶ `Fromkeys(s[,v])`:-create a new dictionary with keys from sequence 's' and values all set to 'v'
- ▶ `Get(k[,v])`:- Return the value associated with key k if key is not found it returns v
- ▶ `Pop(k[,v])`:- Remove the key and its value from dictionary and returns the value if key is not found then the value v is returned

Functions

- ▶ A function is similar to a program that consists of a group of statements that are intended to perform a specific task.
- ▶ The main purpose of a function is to perform a specific task or work
- ▶ When there are several tasks to be performed the programmer will write several functions
- ▶ There are several built in functions in python to perform various tasks.
- ▶ For example:- `print()`, `sum()`, `sqrt()` etc

Advantages of Functions

- ▶ Functions are important in programming because they are used to process data, make calculations or perform any task which is required in the software development
- ▶ Once a function is written, it can be reused as and when required. So functions are reusable code. Because of this reusability
- ▶ Functions provide modularity for programming. A module represents a part of the program. Usually, a programmer divides the main task into smaller sub tasks called modules.
- ▶ Code maintenance will become easy because of functions. When a new feature has to be added to the existing software, a new function can be written and integrated into the software.
- ▶ When there is an error in the software, the corresponding function can be modified without disturbing the other function in the software. Thus code debugging will become easy.
- ▶ The use of function in a program will reduce the length of the program

Difference between a function and method

- ▶ A function contains a group of statements and perform a specific task. A function can be written individually in a python program. A function is using its name.
- ▶ When a function is written inside a class it becomes a method. A method is called using the class name or class objects
- ▶ For example:- `objectname.methodname()`
- ▶ `Classname.methodname()`

Defining a function

- ▶ We can define a function using the keyword `def` followed by function name.
- ▶ After the function name, we should write `parentheses()` which may contain parameters.
- ▶ `def fucntionname(param1,param2....):`
- ▶ `"""doctsring"""`
- ▶ `function statements`

Calling a function

- ▶ A function cannot run on its own . It runs only when we call it. So the next step is to call the function using its name. While calling the function, we should pass the necessary values to the function in the parentheses as:
- ▶ `Add(5,2)`

Returning results from a function

- ▶ We can return the result or output from the function using a 'return' statement in the body of the function.
- ▶ For example, `return c` # return c value out of the function
- ▶ `return 100`
- ▶ `return lst`

Returning Multiple values from a function

- ▶ A function returns a single value in the programming language like C or Java. But in python, a function can return multiple values. When a function calculates multiple results and wants to return the results, we can use the return statement as:
- ▶ `return a, b, c`

Functions are first class objects

- ▶ In Python, functions are considered as first class objects. It means we can use functions as perfect objects. In fact we create a function, the python interpreter internally creates an object
- ▶ Since function are object we can a function to another function just like we can pass an object (or value) to a function. Also, it is possible to return a function from another function

Following possibilities with python functions

- ▶ It is possible to assign a function to a variable
- ▶ It is possible to define one function inside another function
- ▶ It is possible to pass a function as parameter to another function
- ▶ It is possible that a function can return another function

Pass by object reference

- ▶ In python values are sent to function by means of object reference. Everything is considered is an object

Formal and Actual Arguments

- ▶ When a function is defined, it may have some parameters. These parameters are useful to receive values from outside of the function. They are called formal arguments
- ▶ When we call the function, we should pass data or values to the function. These values are called “actual arguments

Types of actual arguments

- ▶ Positional arguments
- ▶ Keyword arguments
- ▶ Default arguments
- ▶ Variable length arguments

Positional arguments

- ▶ These are the arguments passed to a function in correct positional order.
- ▶ The numbers of arguments and their positions in the function definition should match exactly with the number and position of the argument in the function call

Keyword arguments

- ▶ Keyword arguments are arguments that identify the parameters by their names.
- ▶ `def add(a,b):`
- ▶ `add(a=6,b=7)`

Default Arguments

- ▶ We can mention some default value for the function parameters in the definition
- ▶ Def add(a,b=9):

Variable Length Arguments

- ▶ Sometimes, the programmer does not know how many values a function may receive, In that case, the programmer cannot decide how many arguments to be given in the function definition.
- ▶ `Def add(farg, *args)`
- ▶ Farg is the formal argument and *args represent variable length argument

Keyword variable length argument

- ▶ A keyword variable length argument is an argument that can accept any number of values provided in the format of keys and values.
- ▶ `Def message(farg, **kwargs)`
- ▶ `**kwargs` is called keyword variable length argument. This argument internally represent a dictionary object

Local and Global Variables

- ▶ When we declare a variable inside a function, it becomes a local variable
- ▶ A local variable is a variable whose scope is limited only to that function where it is created
- ▶ The local variable is available only in that function and not outside of that function

- ▶ When a variable is declared above a function. It becomes a global variable. Such variable are available to all the functions which are written after it

Global keyword

- ▶ Sometimes, the global variable and the local variable may have the same name. In that case the function, by default, refers to the local variable and ignores the global variable
- ▶ So, the global variable is not accessible inside the function but outside of it. It is accessible
- ▶ When the programmer wants to use the global variable inside a function, he can use the keyword 'global' before the variable in the beginning of the function body

Globals() function

- ▶ The `globals()` function is a built in function which returns a table of current global variable in the form of a dictionary. Hence using this function we can refer to the global variable as `globals()['a']`. Now this value can be assigned to another variable

Anonymous Functions or Lambda Functions

- ▶ A function without a name is called 'anonymous functions'
- ▶ Anonymous function are not defined using def
- ▶ They are defined using the keyword lambda and hence they are called lambda function
- ▶ Lambda argument_list: expression
- ▶ Def square (x):
 return x*x
- ▶ Same function can be written as lambda
- ▶ Lambda x:x*x
- ▶ Lambda function return a function and hence they should be assigned to a function
- ▶ f=lambda x:x*x
- ▶ Value=f(10)

filter function

- ▶ The filter function is useful to filter out the elements of a sequence depending on the result of the function
- ▶ Syntax:
- ▶ `Filter(function,sequence)`
- ▶ 'Function' represent a function name that may return either True or False and sequence represent a list, string or tuple
- ▶ The function is applied to every element of the sequence and when the function return True, the element is extracted otherwise it is ignored

Map function

- ▶ The map function is similar to filter() function but it acts on each element of the sequence and perhaps changes the element
- ▶ The 'function' performs a specified operation on all the elements of the sequence and the modified elements are returned which can be stored in another sequence

Reduce() function

- ▶ The reduce() function reduces a sequence of elements to a single value by processing the elements according to a function supplied
- ▶ Syntax:
- ▶ `Reduce(function,sequence)`